Scan & Resolve

document!
('main.typ')
Proc macro
invocation

Read Cargo.toml
metadata
template-dir

Scan .typ files for
#import
'@namespace/name:version'

In local
data dir?

No

Yes

In cache dir?

Yes

No

Use local
package

Use cached
package

Download from
Typst Universe

Yes

Scan resolved
package:
.typ files for
#import +
typst.toml for
dependencies

More packages
to resolve?

No

All packages
resolved

Compress & Deduplicate

Read from
Cargo.toml:
template-dir,
fonts-dir,
compression-
level

template-dir:
read all files

fonts-dir:
read ttf, otf, ttc
only

resolved
packages:
read files from
each required
package

For each file:
compute BLAKE3
hash

Hash exists
in memory?

Yes

No

Duplicate:
reference
the same blob

Compressed in
previous build?

Yes

No

Load
compressed data
from disk cache
(target/typst-
bake-cache/)

Compress with
zstd
Save as
{hash}_{level}.zst

Store blob in
BTreeMap
(reproducible
ordering)

Macro Expansion

Embed
compressed
blobs
as static byte
arrays in Rust
code

Reconstruct
templates,
fonts, packages
as directory
trees
referencing
blobs

Create
Document struct
that holds
all directories
as read-only
data

rustc compiles
everything
into a self-
contained binary

Runtime

(Optional)
add_file()
Inject runtime
files

Call to_pdf() /
to_svg() /
to_png()

Decompress
entry file

Create
EmbeddedResolver
remaining files
stay compressed

Pass entry,
fonts, resolver
to Typst engine

Typst compiles
document
(files and fonts
decompressed
on demand)

Output PDF /
SVG / PNG